CHAPTER 12

Amortization and Amortized Analysis

EECS 281 study notes

How to read these notes. Anything inside an amber dashed box marked NOT TESTED is not covered in this class. It is included for completeness only. Everything outside those boxes is fair game. The single method you are required to know is **aggregate analysis (12.2.1)**.

12.1 Amortized Complexity

The rent analogy

- Rent is $1,500 a month, paid on the 1st.
- **Worst-case daily cost** = $1,500, because that is what you may have to pay on a given day.
- That number is misleading: the $1,500 buys you the whole month, not just that one day.
- Spreading the payment over all days it covers gives a more meaningful number: $1,500 / 30 days = **$50 per day**. This spread-out number is the **amortized cost**.

Why this matters: `std::vector::push_back()`

- Worst-case time complexity of a single `push_back()` is **$\Theta(n)$**.
- Reason: if the vector is at capacity, the push must
  1. allocate a new array with **double** the capacity,
  2. copy all existing elements over,
  3. then append the new element.
- Key observation: that one expensive push guarantees the **next n pushes are $\Theta(1)$**, since there is now spare capacity.

A vector tracks three pointers: `begin`, `end_sz` (one past last element), `end_cap` (one past capacity).
Push with room left: [1|2|3|4|_|_] → [1|2|3|4|**5**|_] = $\Theta(1)$
Push at capacity: [1|2|3|4] → allocate [_|_|_|_|_|_|_|_] → copy 1,2,3,4 → append 5 = $\Theta(n)$

The problem with using worst case

- Worst case is normally a fine upper bound, but here it is **overly pessimistic**.
- Naive reasoning: "n pushes, each $\Theta(n)$ worst case → $\Theta(n^2)$".
- This bound is far too high, because it is **impossible** for all n pushes to reallocate.
- We need a way to get a **tighter bound on a whole sequence** of operations.

DEFINITIONS

- **Amortization:** spreading the work of rare, expensive steps across many cheap ones.
- **Amortized analysis:** the technique of analyzing a long sequence of operations to get a tighter complexity bound.
- **Amortized complexity:** the resulting per-operation cost you can expect if you run the operation many times, even if some individual calls are expensive.

General procedure

1. Consider a sequence of n operations all at once.
2. Find the **maximum total work** required to complete that whole sequence.
3. Distribute (divide) that total work over all n operations.

The result tells you what an operation truly costs *within a sequence*, without worrying about which individual calls hit the worst case.

12.2 Amortized Analysis Techniques

12.2.1 Aggregate Analysis (THE METHOD YOU NEED TO KNOW)

THE WHOLE METHOD IN TWO STEPS

1. Prove that a sequence of n operations requires at most $T(n)$ work in the worst case.
2. Amortized cost of each operation = $\boldsymbol{T(n)}$ / $\boldsymbol{n}$.

Note: every operation in the sequence gets the *same* amortized cost with this method.

Example A: `push_back()` with capacity doubling

- No reallocation needed → append in constant time, cost 1.
- Reallocation needed → copy all existing elements, then append.
- Capacity doubles, so copies happen on pushes 2, 3, 5, 9, 17, ... copying 1, 2, 4, 8, 16 elements.

Item #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Cost	1	1+1	1+2	1	1+4	1	1	1	1+8	1	1	1	1	1	1	1	1+16

Push 2 costs 1+1 = 2: one unit to append the new element, one to copy the old value over.

Total work:

$$T(n) = (1 + 1 + 1 + \ldots) \; + \; (1 + 2 + 4 + 8 + \ldots + n')$$

left group = cost of appending new elements (done n times) | right group = cost of copying during reallocation
n' = the largest power of two smaller than n

- Appending term: 1 added n times = $\boldsymbol{n}$.
- Copying term: use the sum of a geometric series.

$$\Sigma_{k=0}^{n-1} (ar^k) = a \cdot \frac{1 - r^n}{1 - r} \quad (a = \text{first term}, r = \text{common ratio}, n = \text{number of terms})$$

$$\Sigma_{k=0}^{(\log_2 n+1)-1} 2^k = \frac{1 - 2^{\log_2 n+1}}{1 - 2} = \frac{1 - 2^{\log_2 n} \times 2}{1 - 2} = \frac{1 - 2n}{1 - 2} = \frac{2n - 1}{2 - 1} = 2n - 1$$

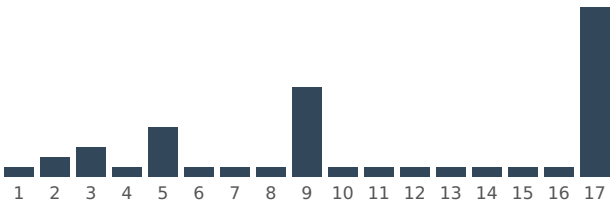
$$T(n) = n + (2n - 1) = 3n - 1 = \Theta(n)$$

RESULT

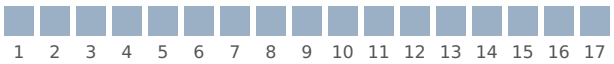
$$\text{Amortized Complexity} = \frac{T(n)}{n} = \frac{\Theta(n)}{n} = \boldsymbol{\Theta(1)}$$

Two things this tells us:

1. With capacity doubling, calling `push_back()` n times costs $n \times \Theta(1) = \boldsymbol{\Theta(n)}$ total, no matter how many pushes are made.
2. Since n pushes are bounded by $\Theta(n)$, each push **inside a sequence** can be treated as if it were a $\Theta(1)$ operation.



Actual work per push. Spikes at 2, 3, 5, 9, 17 are reallocations.



Same total work (48 units for 17 pushes), redistributed evenly. Each push now looks constant.

IMPORTANT NUANCE

This even redistribution is **not** how `push_back()` actually behaves internally. Some calls really do cost more. But for complexity analysis of a *sequence*, treating every push as Θ(1) gives the same result, which is exactly what "amortized Θ(1)" means: the average cost per operation in the sequence is bounded above by Θ(1).

Example B: `push_back()` with growth by a fixed constant *k*

- Suppose capacity grows by a fixed constant k instead of doubling.
- Then reallocations copy k, then $2k$, then $3k$, and so on.

$$T(n) = (1 + 1 + 1 + \ldots) + (k + 2k + 3k + \ldots + \lfloor n/k \rfloor k) \;=\; (1 + 1 + \ldots) + k(1 + 2 + 3 + \ldots + \lfloor n/k \rfloor)$$

Use the sum of an arithmetic sequence (n = number of terms, a_1 = first term, a_n = last term):

$$S_n = \frac{n(a_1 + a_n)}{2} \quad \rightarrow \quad 1 + 2 + \ldots + \lfloor n/k \rfloor = \frac{\lfloor n/k \rfloor (1 + \lfloor n/k \rfloor)}{2} = \Theta(n^2)$$

$$T(n) = n + \Theta(n^2) = \Theta(n^2) \quad \rightarrow \quad \text{Amortized} = \frac{\Theta(n^2)}{n} = \boldsymbol{\Theta(n)}$$

TAKEAWAY

`push_back()` is amortized Θ(1) **only because capacity grows by a multiplicative factor**. Growing by a fixed additive constant gives amortized Θ(n). Any constant multiplier works (1.5, 2, 3, ...), since the geometric series still sums to Θ(n).

NOT TESTED

12.2.2 The Accounting Method

- Assign a **monetary amortized cost** to each type of operation in a sequence.
- If the amortized cost charged exceeds the actual cost, the difference is saved as **credit**.
- Goal: pick prices so the sequence can finish **without ever running out of credit** (balance never goes negative).
- Unlike aggregate analysis, **each operation type can have its own amortized cost**.

Notation: c_i = actual cost of operation i; $\hat{c}$ = amortized cost.

$$\Sigma_{i=1}^{n} c_i \;\leq\; n\,\hat{c} \qquad \textit{(all operations same type)}$$

$$\Sigma_{i=1}^{n} c_i \;\leq\; \Sigma_{i=1}^{n} \hat{c}_i \qquad \textit{(different operation types)}$$

Meaning: total credit collected must be at least the total actual cost incurred.

Applied to `push_back()`: charge $3 per push

- Elementary cost: moving one element (either appending it or copying it) costs $1.

Push	Realloc?	Actual cost	Charged	Balance after
1	no	$1	$3	$2
2	yes (cap→2)	$2 (1 new + 1 copy)	$3	$3
3	yes (cap→4)	$3 (1 new + 2 copies)	$3	$3
4	no	$1	$3	$5
5	yes (cap→8)	$5 (1 new + 4 copies)	$3	$3
6, 7, 8	no	$1 each	$3 each	$9
9	yes (cap→16)	$9 (1 new + 8 copies)	$3	$3

- Copying costs are always paid from surplus earned on earlier pushes.
- The balance resets to $3 after every reallocation and never drops below zero.
- **Where the $3 goes:** $1 to append the element when inserted, $1 to move that same element during its first reallocation, $1 to move some previous element during that reallocation.

- $3 is a constant, so the amortized complexity is **Θ(1)**.

When to use it: best suited for proving a Θ(1) amortized bound. Set elementary operations to $1, then find a constant price that keeps the balance non-negative for any n calls.

NOT TESTED

12.2.3 The Potential Method

- Like the accounting method, it tracks prepaid work, but as **"potential energy"** of the data structure rather than a credit balance.
- Potential energy = work already paid for in the analysis but not yet used.
- Advantage: a function lets you compute stored energy at **any point in time**, after any sequence of operations.

States: D_0 = initial state, D_1 = after operation 1, ..., D_n = after operation n.

Requirements on the potential function Φ:

- $\Phi(D_0) = 0$
- $\Phi(D_i) \geq 0$ for all $1 \leq i \leq n$

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

- Any valid Φ works, but choose one that makes $\hat{c}_i$ as small as possible.
- Ideally ΔΦ is **positive for cheap operations** (build up energy) and **negative for expensive ones** (spend it).

Why it is valid

$$\Sigma\, \hat{c}_i = c_1 + \Phi(D_1) - \Phi(D_0) + c_2 + \Phi(D_2) - \Phi(D_1) + \ldots + c_n + \Phi(D_n) - \Phi(D_{n-1})$$

$$= \Sigma\, c_i + \Phi(D_n) \;\geq\; \Sigma\, c_i \quad \textit{(the terms telescope, and } \Phi(D_n) \geq 0\textit{)}$$

So total amortized cost is always an upper bound on total actual cost, which makes it valid to quote the amortized cost as the complexity bound.

Applied to `push_back()` with doubling

Use $\Phi(D_i) = 2n_i - k_i$, where n_i = size after push i and k_i = capacity after push i. This is valid because Φ is always non-negative.

Case 1, no reallocation (actual cost $c_i = 1$):

$$\hat{c}_i = 1 + (2n_i - k_i) - (2(n_i - 1) - k_i) = 1 + 2 = \mathbf{3}$$

Case 2, reallocation (actual cost $c_i = n_i$: copy $n_i - 1$ elements plus 1 insertion; capacity before = $n_i - 1$, capacity after = $2(n_i - 1)$):

$$\hat{c}_i = n_i + (2n_i - 2(n_i - 1)) - (2(n_i - 1) - (n_i - 1)) = 2 + 2 - 1 = \mathbf{3}$$

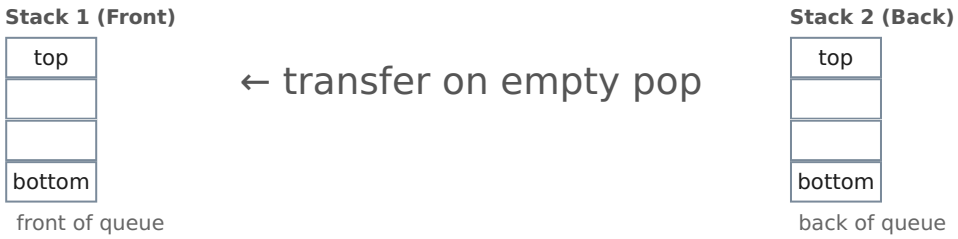
Both cases give a constant 3, so the amortized complexity is **Θ(1)**.

> **REMARK**
> $\Phi(D_0) = 0$ is only a convention. Any Φ with $\Phi(D_i) \geq \Phi(D_0)$ for all i is valid, because you can define $\Phi'(D_i) = \Phi(D_i) - \Phi(D_0)$, which satisfies both conditions and produces identical amortized costs (the $\Phi(D_0)$ terms cancel in the difference).

12.3 Applications of Amortized Analysis

12.3.1 Implementing a Queue with Two Stacks

- One stack is the **front** of the queue, the other is the **back**.
- `push()`: push onto the back stack.
- `pop()`: pop from the front stack. If the front stack is empty, first move **every** element from the back stack to the front stack (which reverses the order, restoring FIFO), then pop.




```
void push(T x) {
  stackBack.push(x);
} // push()

void pop() {
  if (stackFront.empty()) {
    while (!stackBack.empty()) {
      stackFront.push(stackBack.top());
      stackBack.pop();
    } // while
  } // if
  stackFront.pop();
} // pop()
```

- **Worst-case** `pop()` is $\Theta(n)$, when the front stack is empty and the entire back stack must be transferred.
- That case is rare, so expecting n pops to cost $\Theta(n^2)$ is too pessimistic. It is impossible for every pop to be $\Theta(n)$.

Aggregate analysis

- Follow the **lifetime of a single element**. Its total cost is at most **4**:
  1. 1 to push into the back stack
  2. 1 to pop out of the back stack
  3. 1 to push into the front stack
  4. 1 to pop out of the front stack
- So any sequence of n operations costs at most $4n$.

$$Amortized = \frac{T(n)}{n} = \frac{4n}{n} = \boldsymbol{\Theta(1)}$$

NOT TESTED

Accounting method for the two-stack queue

- Actual cost of one stack push or pop = \$1.
- Charge **\$3 per `push()`** and **\$1 per `pop()`**.
- Each initial push costs \$1 but is charged \$3, banking \$2 of credit.
- Moving an element from back to front costs exactly \$2 (\$1 pop from back, \$1 push to front), which is exactly the banked credit.
- The final \$1 to pop out of the front stack is paid by the \$1 amortized cost of `pop()`.
- Both charges are constants, so both operations are amortized **$\Theta(1)$**.

Potential method for the two-stack queue

Use $\Phi(D_i) = 2n_i$, where n_i = number of elements in the **back** stack. Valid since $\Phi(D_0) = 0$ and Φ is never negative.

- **push():** $\hat{c}_i = 1 + 2n_i - 2(n_i - 1) = 1 + 2 = \mathbf{3}$
- **pop(), front stack filled:** cost 1, back stack unchanged → $\hat{c}_i = 1 + 2n_i - 2n_i = \mathbf{1}$
- **pop(), front stack empty:** actual cost = $2n_i + 1$ (pop n_i off back, push n_i onto front, then 1 final pop). Afterwards the back stack is empty so $\Phi = 0 \rightarrow \hat{c}_i = (2n_i + 1) + 0 - 2n_i = \mathbf{1}$

push is constant 3, pop is constant 1, so both are amortized $\Theta(1)$.

12.3.2 Analyzing a Binary Counter

- A k-bit counter stored in a k-element binary array `A`, initialized to 0.
- `.increment()` adds 1 to the binary number.
- Flipping a single bit (0→1 or 1→0) costs 1 unit of work.

A[0]	...	A[k-4]	A[k-3]	A[k-2]	A[k-1]	decimal	cost
0	...	0	0	0	0	0	–
0	...	0	0	0	1	1	1
0	...	0	0	1	0	2	2
0	...	0	0	1	1	3	1
0	...	0	1	0	0	4	3
0	...	0	1	0	1	5	1
0	...	0	1	1	0	6	2
0	...	0	1	1	1	7	1
0	...	1	0	0	0	8	4

Binary refresher: 0=0, 1=1, 2=10, 3=11, 4=100, 5=101, 6=110, 7=111, 8=1000.

- **Worst case for a single increment is $\Theta(k)$**, when all k bits flip (going from $2^{k-1}-1$ to 2^{k-1}).
- $\Theta(nk)$ for n increments is not tight, since all n calls cannot be $\Theta(k)$.

Aggregate analysis

- Not every bit flips on every call:
  - last bit `A[k-1]`: flips every increment
  - `A[k-2]`: flips every 2nd increment
  - `A[k-3]`: flips every 4th increment
  - `A[k-4]`: flips every 8th increment
  - in general the nth-to-last bit flips once every 2^{n-1} increments

$$T(n) = n + \lfloor n/2 \rfloor + \lfloor n/4 \rfloor + \lfloor n/8 \rfloor + \ldots + \lfloor n/2^{k-1} \rfloor$$

$$= \Sigma_{i=0}^{k-1} \lfloor n/2^i \rfloor \leq n \Sigma_{i=0}^{k-1} (1/2)^i = 2n(1 - 1/2^k) < 2n$$

- n increments involve fewer than $2n$ bit flips, so $T(n) = \Theta(n)$.

$$Amortized = \frac{\Theta(n)}{n} = \boldsymbol{\Theta(1)}$$

NOT TESTED

Accounting method for the binary counter

- Charge **$2 per `.increment()`**.
- $1 pays for the flip 0→1 right now; the other $1 is left **on that bit** to pay for its eventual flip back 1→0.
- So every bit currently holding a 1 carries $1 of credit; when it flips to 0 its credit drops to $0.

- The balance never goes negative, and \$2 is a constant, so `.increment()` is amortized Θ(1).

Example trace of balance: 000→\$0 | 001→\$1 | 010→\$1 | 011→\$2 | 100→\$1 | 101→\$2 | 110→\$2 | 111→\$3 | 1000→\$1

Potential method for the binary counter

Use $\Phi(D_i) = n_i$, where n_i = number of 1s in the number after increment i.

i	1	2	3	4	5	6	7	8	9
c_i	1	2	1	3	1	2	1	4	1
$\Delta\Phi$	1−0	1−1	2−1	1−2	2−1	2−2	3−2	1−3	2−1
$\hat{c}_i$	2	2	2	2	2	2	2	2	2

General proof. Let m_i = number of bits flipped from 1 back to 0 on increment i. Exactly one bit always flips 0→1, so:

- actual cost $c_i = m_i + 1$
- change in potential = $1 - m_i$ (one more 1, minus the m_i that became 0)

$$\hat{c}_i = (m_i + 1) + (1 - m_i) = \mathbf{2}$$

Always constant 2, so amortized Θ(1).

12.3.3 Pairing Heap Removal

- Claim from chapter 10: popping the highest priority element from a pairing heap is **amortized $O(\log(n))$**.
- **Worst case for a single pop is $\Theta(n)$**: if every other node is a direct child of the root, popping the root forces you to iterate over and meld Θ(n) subheaps.
- With a **two-pass or multipass** melding approach, that expensive pop leaves the remaining nodes arranged so that future pops are $O(\log(n))$.
- Inserting n elements then popping all n until empty takes $O(n \log(n))$ total, because of this reorganization.

$$Amortized = \frac{O(n\ log(n))}{n} = \boldsymbol{O(log(n))}\ per\ pop$$

root 9 with children 1..8 —> `.pop()` —> pair up (1,2)(3,4)(5,6)(7,8) —> meld pairs —> balanced-ish tree of depth $O(\log n)$

12.4 Amortized vs. Average-Case Analysis

DO NOT CONFUSE THESE

- **Amortized complexity:** cost of an operation when evaluated **over a sequence of operations**.
- **Average-case complexity:** expected cost of a **single call** over **all possible inputs**.

- Average-case relies on **probabilistic assumptions** about the inputs and data structures. You must consider every way the operation could run and compute expected performance over all of them.
- Amortized analysis needs **no such assumptions**. Think of it as a **worst-case analysis for a sequence**: bound the total work for the sequence, then divide by the number of operations.
- A single call can still exceed its amortized cost. Example: worst case Θ(n) with amortized Θ(1) means one call may run in Θ(n), but many calls average out to Θ(1) each.

Aspect	Average-Case Complexity	Amortized Complexity
Methodology	Averages the performance of a single operation over all possible inputs of size n	Finds the worst-case cost over a long sequence of operations, then divides by the number of operations
Assumptions	Relies on probabilistic assumptions about the data structures and inputs involved	Makes no probabilistic assumptions about inputs or operations
Guarantees	Expected running time that holds on average if the input distribution matches the assumed model	Worst-case guarantee for the average cost per operation over a long sequence

Variance risk	Unlucky inputs can perform worse than the computed average	No unlucky sequence can violate the bound; it holds even in the worst-case sequence
Typical use	Algorithms whose performance depends strongly on input distribution	Data structures with occasional expensive restructuring operations that do not reflect typical performance

Quick Reference: Common Amortized Results

Situation	Total work $T(n)$	Amortized per op
`push_back()`, capacity **doubled**	$\Theta(n)$	$\Theta(1)$
`push_back()`, capacity × any constant factor (1.5, 3, ...)	$\Theta(n)$ (geometric series)	$\Theta(1)$, unchanged
`push_back()`, capacity **+ fixed constant** (e.g. +1000)	$\Theta(n^2)$ (arithmetic series)	$\Theta(n)$, worse
Reallocate + `std::sort()` on every reallocation	$\Theta(n \log(n))$	$\Theta(\log(n))$
Op i costs i when i is a power of 2, else 1	(1+1+...) + (1+2+4+...) = $\Theta(n)$	$\Theta(1)$
Queue built from two stacks (push / pop)	$\leq 4n$	$\Theta(1)$ both
Binary k-bit counter `.increment()`	$< 2n$	$\Theta(1)$
Pairing heap `.pop()` (two-pass / multipass)	$O(n \log(n))$	$O(\log(n))$
`std::queue` over `std::list` (no reallocation ever)	$\Theta(n)$	$\Theta(1)$; worst case is also $\Theta(1)$

REASONING PATTERN WORTH MEMORIZING

Many amortized $\Theta(1)$ proofs use the same trick: **bound the total cost by the lifetime of each element**. If every element that enters the structure can be touched only a constant number of times before it leaves for good, total work for n operations is $O(n)$, so each operation is amortized $\Theta(1)$.

Examples of that pattern:

- **Ordered stack** (a push pops off everything larger before inserting): worst case push $\Theta(n)$, pop $\Theta(1)$; each element is pushed once and popped once, so cost $\leq 2n$ and both are amortized $\Theta(1)$. An expensive push also removes many elements, making future pushes cheaper.
- **Batched queue** with `push`, `pop`, `clear(k)`: worst case push and pop $\Theta(1)$, clear $\Theta(n)$; each element costs at most 2 over its lifetime, so all three are amortized $\Theta(1)$.
- **Queue with a `.kick()` that removes every 5th student:** each kick is a $\Theta(n)$ traversal but deletes 1/5 of the elements. Total nodes visited across all kicks is about $5k$ for k removals, and each element is removed at most once, so all operations are amortized $O(1)$. (Alternate view: traversal work is $n + (4/5)n + (4/5)^2 n + \ldots = \Theta(n)$ by the geometric series.)
- **Auto-backup container** that copies everything every kth insertion: this is the fixed-constant-growth case, so $T(x) = \Theta(x^2)$ and each insert is amortized $\Theta(x)$.

COMMON MISCONCEPTIONS

- Amortized complexity is **never worse** than the operation's own worst case; it is at least as good.
- Amortized analysis is **most useful** when the worst case is much worse than typical performance but happens rarely. It is useless when average and worst case are already identical.
- Amortized analysis gives a **tighter upper bound** on a sequence when individual calls have different costs, independent of input.